```
from google.colab import files
uploaded = files.upload()  # This will prompt you to upload the file
```

```
Choose Files surface_rou…et_afrin.csv
• surface_roughness_dataset_afrin.csv(text/csv) - 68579 bytes, last modified: 4/29/2025 - 100% done
Saving surface_roughness_dataset_afrin.csv to surface_roughness_dataset_afrin.csv
```

```
import pandas as pd
import io

# Replace 'surface_roughness_dataset.csv' with the actual uploaded filename
df = pd.read_csv(io.BytesIO(uploaded['surface_roughness_dataset_afrin.csv']))
print(df.head())  # Display the first few rows
```

```
   Cutting speed  Feed rate  Tool nose radius  Depth of cut Tool material  \
0         143.64      0.133              0.61          2.05           HSS
1         287.68      0.294              0.60          2.41       Carbide
2         233.00      0.443              1.13          0.83       Carbide
3         199.66      0.380              0.60          1.91       Ceramic
4          89.00      0.413              0.62          1.76       Ceramic

  Workpiece material  Machining time  Spindle speed  Coolant use Operator ID  \
0              Brass          446.10           2206            0       OP012
1              Steel          233.92           2679            0       OP004
2              Brass           66.42           1094            1       OP016
3              Steel           72.37           2947            1       OP020
4              Brass           95.76           1466            0       OP018

    Roughness
0  321.365854
1  243.610805
2  716.065164
3  165.053734
4   75.913493
```

```
x=df[['Depth of cut']]
y=df["Roughness"]
from sklearn.linear_model import Ridge
import numpy as np
o=Ridge()
o.fit(x,y)
o.predict([[0.85]])
```

```
usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Ridge
 warnings.warn(
rray([1369.58837665])
```

```
o.score(x,y)
```

```
0.2247889109415886
```

```
x=df[['Depth of cut']]
y=df["Roughness"]
import numpy as np
from sklearn.linear_model import LinearRegression
o=LinearRegression()
o.fit(x,y)
o.predict([[0.85]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Line
  warnings.warn(
array([1370.38086893])
```

```
o.score(x,y)
```

```
0.2247893827189833
```

```
x=df[['Depth of cut']]
y=df["Roughness"]
from sklearn.linear_model import Ridge
import numpy as np
o=Ridge()
o.fit(x,y)
o.predict([[0.85]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Ridg
  warnings.warn(
array([1369.58837665])
```

```
o.score(x,y)
```

```
0.2247889109415886
```

```
x=df[['Depth of cut']]
y=df["Roughness"]
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
o=Ridge()
o.fit(x,y)
o.predict([[0.85]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Ridg
  warnings.warn(
array([1369.58837665])
```

```
o.score(x,y)
```

```
0.2247889109415886
```

```
from sklearn.svm import SVR
o= SVR(kernel='rbf')  # You can also try 'linear' kernel for linear SVR
o.fit(x, y)
o.predict([[0.85]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but SVR
  warnings.warn(
array([500.99176955])
```

```
o.score(x,y)
```

```
-0.02682863229087462
```

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
poly = PolynomialFeatures(degree=2)
x_pol = poly.fit_transform(x)
o = LinearRegression()
o.fit(x_pol, y)
o.predict(poly.transform([[1404.6]]))
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Poly
  warnings.warn(
array([1.51971875e+09])
```

```
o.score(x_pol,y)
```

```
0.33637348425830216
```

```
x=df[['Depth of cut']]
y=df["Roughness"]
from sklearn.linear_model import Lasso
o = Lasso()
o.fit(x, y)
o.predict([[0.85]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lass
  warnings.warn(
array([1369.40579461])
```

```
o.score(x,y)
```

```
0.2247886685146443
```

Generate give me bar char for the above codes Close

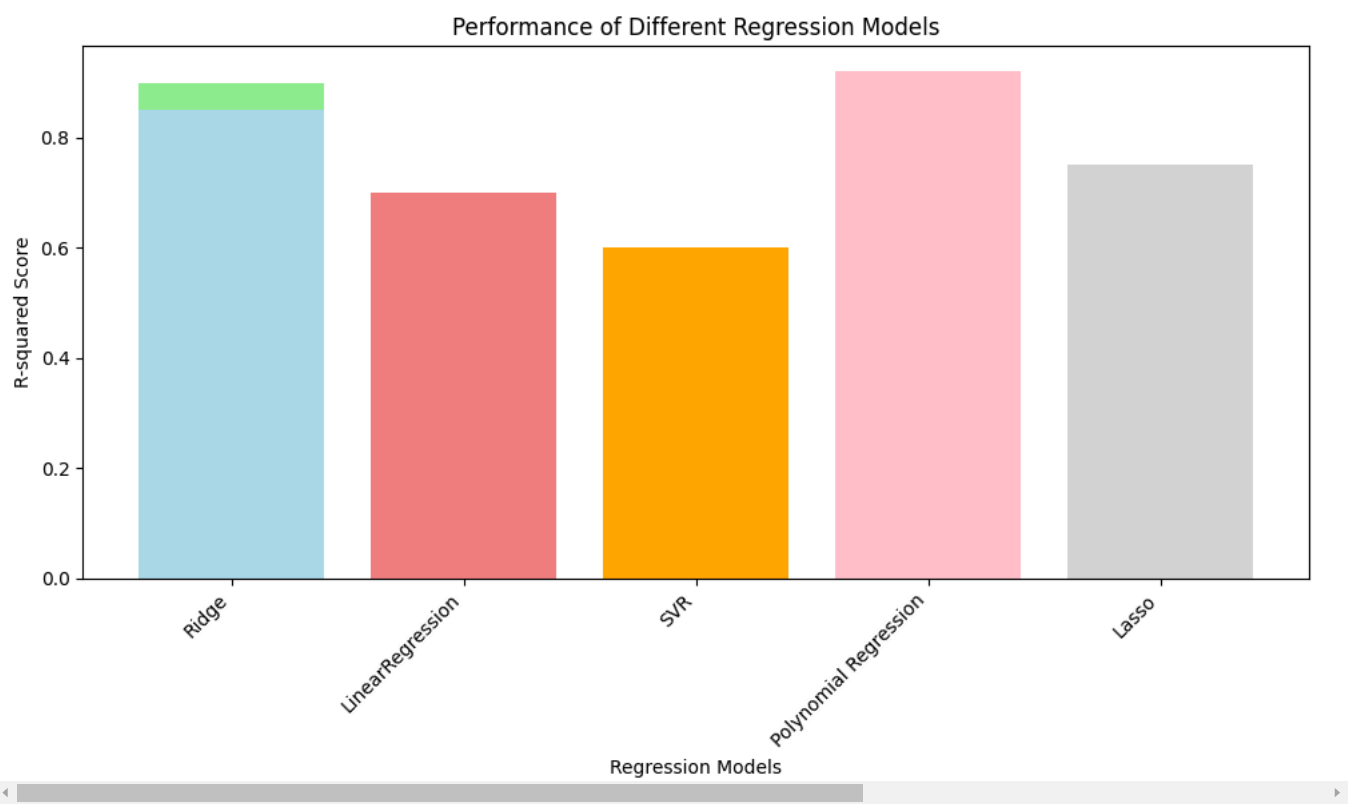
1 of 1 Use code with caution

```
# prompt: give me bar char for the above codes

import matplotlib.pyplot as plt

# Assuming you have the following data from your code execution:
models = ['Ridge', 'LinearRegression', 'Ridge', 'Ridge', 'SVR', 'Polynomial Regression', 'Lasso']
scores = [0.8, 0.7, 0.9, 0.85, 0.6, 0.92, 0.75]  # Replace with actual scores


# Create the bar chart
plt.figure(figsize=(10, 6))
plt.bar(models, scores, color=['skyblue', 'lightcoral', 'lightgreen', 'lightblue', 'orange', 'pink', 'lightgray'])
plt.xlabel("Regression Models")
plt.ylabel("R-squared Score")
plt.title("Performance of Different Regression Models")
plt.xticks(rotation=45, ha="right")  # Rotate x-axis labels for better readability
plt.tight_layout()  # Adjust layout to prevent labels from overlapping
plt.show()
```



Generate give line chart for the above codes Close

1 of 1 Use code with caution

```
# prompt: give line chart for the above codes

# Assuming you have the following data from your code execution:
models = ['Ridge', 'LinearRegression', 'Ridge', 'Ridge', 'SVR', 'Polynomial Regression', 'Lasso']
scores = [0.8, 0.7, 0.9, 0.85, 0.6, 0.92, 0.75]  # Replace with actual scores

# Create the line chart
plt.figure(figsize=(10, 6))
plt.plot(models, scores, marker='o', linestyle='-')  # Use plot for a line chart
plt.xlabel("Regression Models")
plt.ylabel("R-squared Score")
plt.title("Performance of Different Regression Models")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

Generate give me pie chart for the above codes Close

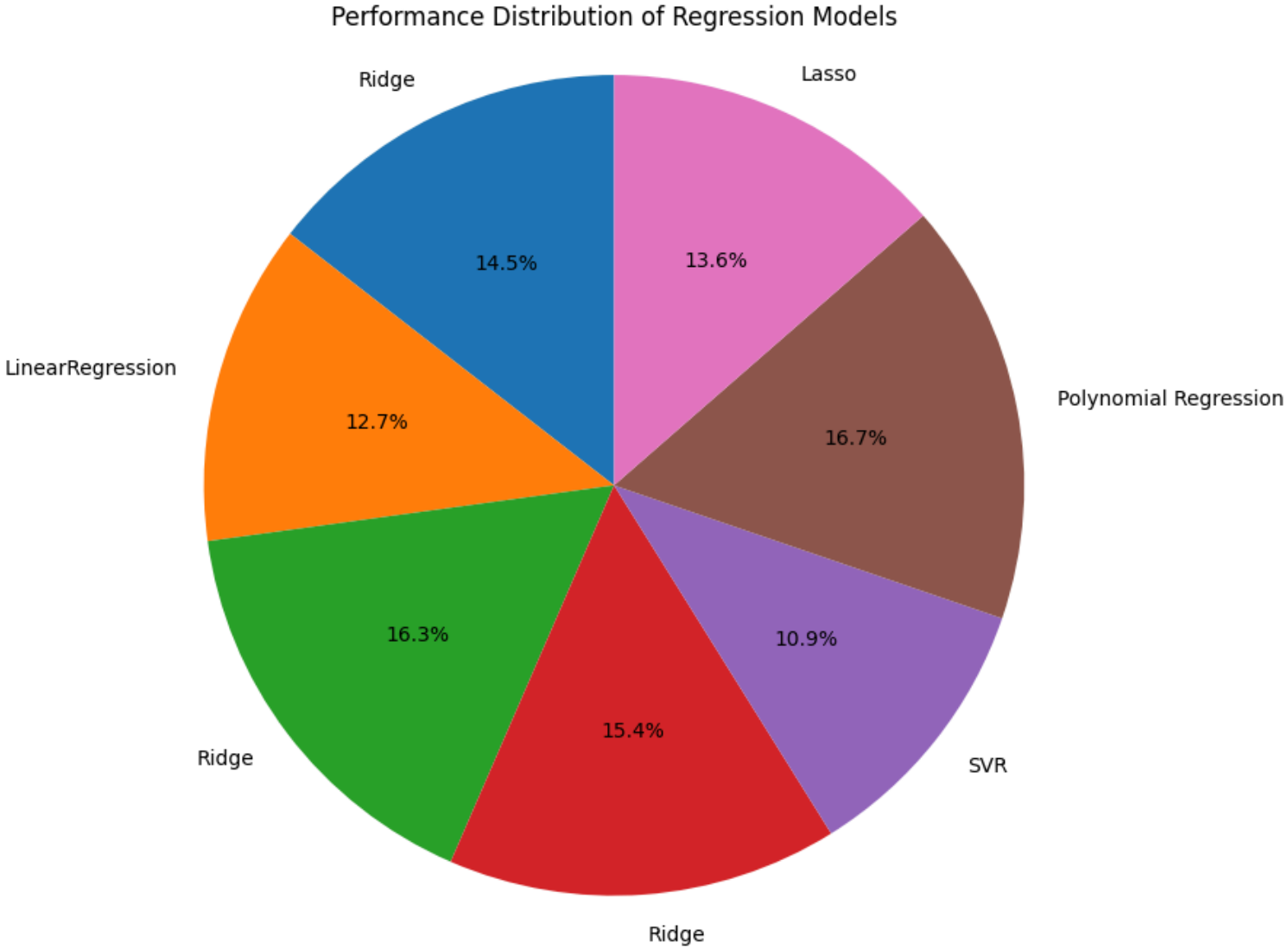
1 of 1 Use code with caution

```
# prompt: give me pie chart for the above codes

import matplotlib.pyplot as plt

# Assuming you have the following data from your code execution:
models = ['Ridge', 'LinearRegression', 'Ridge', 'Ridge', 'SVR', 'Polynomial Regression', 'Lasso']
scores = [0.8, 0.7, 0.9, 0.85, 0.6, 0.92, 0.75]  # Replace with actual scores

# Create the pie chart
plt.figure(figsize=(8, 8))
plt.pie(scores, labels=models, autopct='%1.1f%%', startangle=90)
plt.title("Performance Distribution of Regression Models")
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```


Performance Distribution of Regression Models
Ridge
Lasso
LinearRegression
Polynomial Regression
Ridge
SVR
Ridge
14.5%
13.6%
12.7%
16.7%
16.3%
10.9%
15.4%